

TERMINAL TASK — User Guide (v0.3.0)

A tiny always-on-top task HUD for Windows with a terminal soul. It lives as a small glowing ball at the edge of your screen; open it, type, done. Built for 2-second capture — no accounts, no cloud, your data never leaves your machine.

1. The Ball

When collapsed, the widget is a 60×60 floating ball docked at the screen edge. It tells you three things at a glance:

- **A number** — how many main tasks are still open today.
-  — everything's done.
-  — empty day, nothing loaded.
- **Glow color** — green when relaxed, shifting toward red over the last 2 hours before your deadline. Red means overdue.

Click the ball to open the panel. Drag it to reposition. The tray icon does the same jobs: left-click toggles the panel, right-click gives Open / Hide / Reset Position / Quit.

Global hotkey:  (default) shows/hides the widget from anywhere. Rebind it with .

2. The Panel

The panel is one list and one prompt. The header shows your deadline and a live countdown ( · ). The progress bar () counts every leaf item, so ticking off subtasks visibly moves it.

Everything you do flows through the  prompt at the bottom:

- Type text +  → new task.
- Type  → command palette pops up with ghost autocomplete (press  or  to accept the suggestion).
- Hover any row → a small  chip appears at its right edge; hover the chip to expand the action buttons (). The chip stays tiny so long task text remains readable. Buttons that would be illegal on that row simply aren't drawn — no  on a third-level row, no  on a top-level task.
- The prompt changes color to tell you what mode you're in: **amber**  while editing, **cyan**  while chaining subtasks.

The panel auto-collapses when your mouse leaves it (unless you're typing).  also steps you back: cancel mode → deselect → collapse.

3. Working with Tasks

Adding

Input	Result
 + Enter	New task (it flashes and scrolls into view)

Input	Result
<code>> some text</code> or <code>- some text</code>	Child under the selected row, at whatever depth it sits (or under the last task if nothing is selected, so <code>task</code> → <code>> sub</code> chains) — it flashes where it lands
<code>/l text</code>	Into the backlog (long-term list at the bottom)
<code>/daily text</code>	Recurring ritual — reseeds as a fresh task every morning

Rapid subtask entry

Select any row (arrows or hover) and press `>` — or click its `[ + ]` button. The prompt turns cyan and every `Enter` chains another child under it. Empty `Enter` or `Esc` exits. A row already at the third level can't host children, so it has no `[ + ]` and `>` answers with `[!] max depth is 3 levels`.

Completing

- **Click** a task's text (or a nested row's marker) to toggle it.
- **Space** toggles the selected item.
- Completing a row completes everything under it, all the way down; completing the last open leaf completes every ancestor above it.

Editing

- **Double-click** any task, subtask, or backlog text — it loads into the prompt (amber `edit>`), `Enter` saves, `Esc` cancels.
- Keyboard: select it and press `Enter`.

Deleting

- Hover → `[ x ]`, or select and press `Backspace` / `Delete`.
- `/clear` wipes today's list (a notice reminds you: `Ctrl+Z` restores).

Reordering — the fun part

- **Carets**: hover a row, hover its `[ ... ]` chip, and click `[ ^ ]` / `[ v ]`. The row *stays put* while a blurred cyan ghost line floats at the exact slot it will land in — keep clicking the same spot to move it further. Tall rows preview as a condensed block (`task text ·3 sub`); short ones as a thin dashed insertion rule, indented to the depth they'll land at. The move commits automatically a moment after your last click (or when you move to another row). `Esc` aborts.
- **Alt + mouse wheel**: hover a row, hold `Alt`, scroll. The row rides the wheel.
- **Alt + ↑/↓**: same thing for keyboard users.
- Reordering is always **among siblings** — a row moves within its own parent and never pops out of it. Whatever is nested under it travels along. Any committed move is a single `Ctrl+Z` step.

Structure

The list nests **three levels deep**, and no further. Depth reads off the indentation, the thin left rule that runs down each nested group, and the row marker: `[1]` `[2]` ... for top-level tasks, `[ ]` at level 2, `[·]` at level 3, `[x]` when done.

- **Tab** (or the `[ » ]` button) demotes the selected row under the row above it — **with everything nested under it, intact**. It needs a row above it at the same level, and the whole subtree has to still fit inside three levels; otherwise it's a no-op with `[!] max depth is 3 levels`.

- `Shift+Tab` (or `[ < ]`) promotes the row one level out, subtree intact, placed right below its old parent. A top-level task has nowhere to go, so it does nothing.
- If a hand-edited `state.json` ever brings in something deeper, it's clamped to three levels on load and you get a `[!] flattened N items...` notice the next time you open the panel.

4. The Backlog

`/l text` drops ideas into the **BACKLOG / LONG-TERM** section below your day list — things you don't want cluttering today. Bare `/l` jumps straight to the section. The header shows the item count, each item shows an age tag (`·3d`, amber at 14+ days), and once items start aging the header adds a nudge: *"2 aging — promote or prune."*

Promote one to today with its `[ > ]` button or by selecting it and pressing `Space` — it flashes at the bottom of your task list so you see where it went.

Lifecycle: backlog items persist across days — the day rollover never touches them; they only leave by being promoted or deleted. The widget applies gentle, escalating pressure as they age:

1. **Day 1+** — a dim `·3d` age tag appears on each item.
2. **Day 7+** — once per day, the oldest such item surfaces with a `⚡ consider today?` prompt: `[ > ]` promotes it, `[ x ]` snoozes the prompt until tomorrow.
3. **Day 14+** — the age tag turns amber and the section header counts them: *"2 aging — promote or prune."*

Editing works like everywhere else (double-click, or select + Enter); note that `Ctrl+Z` does **not** cover backlog deletes yet.

5. Deadline & Theme

The deadline

`/deadline 18:00` sets **today's** deadline — it's one-time, not recurring, and clears automatically at the day rollover. From two hours out, the whole widget's glow slides green → yellow → red; past the deadline the countdown flips to `T+...` in red. One glance at the ball = how much time you have left.

`/deadline off` clears it early (`clear` and `none` work too). Bare `/deadline` tells you what's currently set.

Themes

Three colour presets ship with the widget:

Preset	Identity	Notes
<code>default</code>	green	The original look. Ramps green → red before a deadline.
<code>mono</code>	grey	Zero saturation — the accent colours carry all the meaning. Never ramps , by design: it's the preset for people who don't run deadlines, and a grey UI lurching to red would be incoherent.
<code>ice</code>	blue	Ramps blue → red. Its action accent moves to violet, because a cyan accent would vanish into a blue identity colour.

Three ways to switch, all equivalent:

- **Type it:** `/theme ice`
- **Type a unique prefix:** `/theme i` — `d`, `m`, and `i` are each unambiguous today. If a prefix ever matches two presets, the widget lists them instead of guessing.

- **Pick it:** type `/theme` followed by a space. The dropdown above the input lists every preset with a colour swatch; **click one and it applies immediately** — no Enter needed. The active preset is marked.

Bare `/theme` posts the list as a notice, with a `*` on the current one.

The ramp — the glow that slides toward red over the last two hours — can be turned off independently of the preset: `/theme ramp off`. It stays off when you switch presets, so turning it off once is permanent until you say otherwise. With the ramp off, the header countdown is your urgency signal.

Your choice survives restarts. If the stored preset name is ever unrecognisable, the widget silently falls back to `default` rather than greeting you with an error.

6. Days, History & Streaks

At the first launch of each new day:

- Completed tasks are archived and written to `Documents\TerminalTasks\YYYY-MM-DD.md`.
- Unfinished tasks carry over.
- Your `/daily` rituals reseed as fresh unchecked tasks.
- The deadline resets.

Open `/log` (tab `4:LOG` in the manual) for: your **streak** gauge (consecutive days with ≥ 1 completion — today counts live), a **last-7-days** summary (`23 done · 4 active days · best Tue (8)`), your **daily rituals** list (remove with `[ x ]`), and the last 14 archived days.

7. Commands

Command	What it does
<code>/deadline</code> <code>HH:MM</code>	Today's deadline (one-time, clears at day end) · <code>off</code> clears it
<code>/theme <name></code>	Colour preset — <code>default</code> , <code>mono</code> , <code>ice</code> . A unique prefix works (<code>/theme i</code>); bare <code>/theme</code> lists them
<code>/theme ramp</code> <code>on\ off</code>	Whether the glow ramps toward red before a deadline
<code>/l <text></code>	Add to backlog
<code>/daily <text></code>	Recurring daily ritual
<code>/log</code>	History, streak & rituals
<code>/clear</code>	Clear today's tasks (Ctrl+Z restores)
<code>/export</code>	Snapshot all data to a timestamped JSON
<code>/shortcut</code>	Rebind the global hotkey — live preview while you hold modifiers; Enter saves, bare Enter resets to <code>Alt+X</code> , Esc cancels
<code>/startup [on\ off]</code>	Launch at login. On by default , including after an upgrade or a reinstall — bare <code>/startup</code> toggles, <code>on</code> / <code>off</code> set it explicitly. Turn it off and it stays off, through any later reinstall. Current state is shown in <code>/about</code> .
<code>/help</code> · <code>/about</code>	The manual · version & info

8. Keyboard Reference

Key	Action
↑ / ↓	Navigate every visible row, tasks → nested rows → backlog (wraps)
Space	Toggle complete / promote backlog item
Enter	Edit selected item
>	Rapid child-entry mode under the selected row
Alt+↑/↓	Reorder selected row among its siblings
Tab / Shift+Tab	Demote / promote — subtree moves intact, max 3 levels
Backspace / Del	Delete selected
Ctrl+Z	Undo (tasks, backlog & rituals together, 20 steps) — keyboard only, on purpose
Esc	Cancel mode → deselect → collapse (also aborts a pending move)
→ / Tab	Accept the ghost completion — completes a command, and after a space, its values (<code>/theme i</code> → <code>/theme ice</code>)
Alt+X	Global show/hide (rebindable via <code>/shortcut</code>)

9. Your Data

Everything is local. Working state lives in the app's storage and is mirrored (debounced) to:

```
Documents\TerminalTasks\  
  state.json           ← live mirror; auto-restores if app storage is ever wiped  
  export-*.json        ← /export snapshots  
  YYYY-MM-DD.md        ← daily archive logs (human-readable Markdown)
```

Back up that folder and you've backed up the widget. Delete the app and the folder survives.

10. Tips

- Park the widget on a screen edge; `Alt+X` in, type, `Esc` out — capture in under 2 seconds.
- Use `/daily` for morning rituals and let the streak gauge guilt you kindly.
- The backlog age tags are intentional pressure: amber items are asking to be promoted or deleted.
- If the ball ever seems missing after sleep/monitor changes, it self-heals within seconds; tray → Reset Position is the manual override.